Agile -lab:

TASK 1 – CI/CD Pipeline Setup (Exam Format Answer)

Aim

To implement a Continuous Integration and Continuous Deployment (CI/CD) pipeline for automating build, test, and deployment of an application.

Tools Used

Git

Jenkins

Docker

Prerequisites

Git repository with source code

Jenkins installed and configured

Docker installed

Basic pipeline configuration knowledge

Procedure

Step 1: Create a Git Repository

Initialize repository

Push application code

git init

git add .

git commit -m "Initial commit"

git push origin main

Step 2: Configure Jenkins Job

Open Jenkins dashboard

Create a new Pipeline Job

Connect Git repository

Step 3: Write Jenkins Pipeline Script

```
pipeline {
    agent any

    stages {

        stage('Clone') {
            steps {
                git 'https://github.com/user/repo.git'
            }
        }

        stage('Build') {
            steps {
                sh 'echo Building application'
            }
        }

        stage('Test') {
            steps {
                sh 'echo Running tests'
            }
        }

        stage('Deploy') {
            steps {
                sh 'echo Deploying application'
            }
        }
    }
```

}

Step 4: Build Docker Image

docker build -t myapp .

Step 5: Run Docker Container

docker run -d -p 8080:80 myapp

Step 6: Trigger Pipeline

Run Jenkins job manually or via webhook

Observe pipeline stages execution

Output

Code is automatically:

Cloned

Built

Tested

Deployed

Application runs successfully in Docker container

Result

Successfully implemented a CI/CD pipeline using Jenkins and Docker, enabling automated integration and deployment.

Conclusion

CI/CD pipeline improves:

Development speed

Code quality

Deployment efficiency

Task 2:
Simple Static Web Page using CI & CD with Docker and Kubernetes

Aim

To develop a static web application, implement Continuous Integration and Continuous Deployment (CI/CD) using Jenkins, and deploy the application using Docker and Kubernetes.

Tools Used

GitHub

Jenkins

Docker

Kubernetes

Visual Studio Code

Prerequisites

GitHub account and repository

Jenkins installed and configured

Docker installed

Kubernetes cluster (Minikube / local setup)

kubectl configured

Procedure

Step 1: Create Static Web Page

Create HTML pages including:

Student Results

Rivera Events

Conference Details

Price List

Student Timetable

Example:

```
<!DOCTYPE html>
<html>
<head>
   <title>Student Results</title>
</head>
<body>
   <h1>Student Results</h1>
   <p>Name: John</p>
   <p>Marks: 90%</p>
</body>
</html>
```

Step 2: Push Code to GitHub

```
git init
git add .
git commit -m "Initial commit"
git push origin main
```

Modify code and push updates:

```
git add .
git commit -m "Updated webpage"
git push origin main
```

Step 3: Create Jenkins Pipeline Job

Open Jenkins dashboard

Create new Pipeline Job

Link GitHub repository

Step 4: Write Jenkins Pipeline Script (CI + CD)

```
pipeline {
    agent any

    stages {

        stage('Clone') {
            steps {
                git 'https://github.com/user/repo.git'
            }
        }

        stage('Build Docker Image') {
            steps {
                sh 'docker build -t webapp:v1 .'
            }
        }

        stage('Run Container') {
            steps {
                sh 'docker run -d -p 8080:80 webapp:v1'
            }
        }

        stage('Deploy to Kubernetes') {
            steps {
                sh 'kubectl apply -f deployment.yaml'
                sh 'kubectl apply -f service.yaml'
            }
        }
```

```
  }
}
```

Step 5: Create Dockerfile

```
FROM nginx:latest
COPY . /usr/share/nginx/html
```

Step 6: Build and Push Docker Image

```
docker build -t webapp:v1 .
```

Step 7: Create Kubernetes Deployment File

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: webapp
  template:
    metadata:
      labels:
        app: webapp
    spec:
      containers:
      - name: webapp
        image: webapp:v1
        ports:
        - containerPort: 80
```

Step 8: Create Kubernetes Service File

```
apiVersion: v1
```

```
kind: Service
metadata:
  name: webapp-service
spec:
  type: NodePort
  selector:
    app: webapp
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30007
```

Step 9: Deploy Application in Kubernetes

```
kubectl apply -f deployment.yaml
kubectl apply -f service.yaml
```

Step 10: Verify Deployment

```
kubectl get pods
kubectl get services
```

Access application:

http://<node-ip>:30007

Output

Static web pages successfully created

Code integrated via Jenkins pipeline

Docker image built and deployed

Application running on Kubernetes cluster

Accessible through NodePort

Result

Successfully implemented CI/CD pipeline for a static web application using Jenkins, Docker, and Kubernetes, and deployed the application in a scalable environment.

Conclusion

The experiment demonstrates:

Automated build and deployment using CI/CD

Efficient containerization with Docker

Scalable deployment using Kubernetes